

GitHub AI Code Scanning & Test Synthesizer

Enterprise Multi-Agent System Architecture & Technical Documentation

1. Executive System Overview

The GitHub AI Code Scanning & Test Suite Synthesizer Platform is an autonomous multi-agent AI system designed to ingest, parse, analyze, index, audit, and synthesize comprehensive industrial test suites for any public GitHub repository.

Operating on a 7-Node StateGraph Workflow (powered by LangGraph) and a Graph-RAG Knowledge Base (stored natively in Neo4j), the platform parses Babel AST dependencies, tracks Taint Data Flow, audits OWASP Top 10 vulnerabilities, scans dependency CVEs, detects secret leaks, probes dynamic runtime API routes (DAST), and synthesizes 7 distinct testing categories.

2. Technology Stack & Frameworks Used

- **LangGraph (@langchain/langgraph):** Orchestrates the 7-Node StateGraph Multi-Agent execution pipeline with parallel security branching and state convergence.
- **Babel AST Parser (@babel/parser):** Parses JavaScript and TypeScript source files into Abstract Syntax Trees, walking nodes for functions, imports, routes, and taint sources.
- **Neo4j Cypher Engine (neo4j-driver):** Stores code topological relationships (:File, :Function, :Class, :Endpoint, :Vulnerability) using native Cypher graph queries (bolt://127.0.0.1:7687).
- **SQLite Graph Fallback:** Disk-backed in-memory property graph engine providing zero-crash resilience if Neo4j is offline.
- **Google Gemini 2.5 API:** Generates AI security code audits and context-aware unit test suites under Graph-RAG token budget constraints.
- **Backend Web Server (Express.js):** Node.js REST API server managing asynchronous job submission, status polling, and CORS security.
- **Frontend Web App (Next.js 14):** React-based enterprise dark-mode interactive dashboard with live percentage progress polling.

3. LangGraph Multi-Agent Pipeline Architecture (7 Nodes)

The platform organizes execution into 7 decoupled agent nodes with parallel security branching:

- **Node 1 (ingestRepo):** Streams repository zipball archives from GitHub API and ranks files by topological PageRank centrality.
- **Node 2 (parseAST):** Extracts AST symbols, function signatures, cyclomatic complexity, API routes, and user input taint sources (req.body, req.query, req.params).
- **Node 3 (indexGraph):** Populates Neo4j graph storage with node labels and relationships (:CONTAINS, :EXPOSES_ROUTE, :IMPORTS).
- **Node 4 (runSASTAuditor - Parallel):** Executes 18 OWASP Top 10 SAST rules and tracks Taint Data Flow to dangerous sinks (SQLi, Exec, Path Traversal, SSRF, XSS, NoSQLi).
- **Node 4.5 (runDependencyScan - Parallel):** Queries OSV.dev API (v1/querybatch) to detect known package CVE vulnerabilities.
- **Node 4.6 (runSecretScan - Parallel):** Scans codebase patterns for leaked AWS secret keys, RSA blocks, PATs, Stripe keys, and DB strings.
- **Node 4.7 (runDASTScan - Parallel):** Fires live dynamic HTTP security probes against exposed API routes.
- **Convergence Node (mergeSecurityResults):** Combines parallel findings from SAST, Taint, CVEs, Secrets, and DAST into a single unified security audit.
- **Node 5 (synthesizeTests):** Invokes modular generator modules to synthesize 7 industrial testing categories.

4. 7 Industrial Testing Categories Synthesized

- **1. Unit Testing:** Function parameter boundaries and logic assertions (Jest / PyTest).
- **2. Functional Testing:** End-to-End user workflows and supertest API endpoint suites.
- **3. Integration Testing:** Cross-module interaction test suites derived from Neo4j import graph.
- **4. Regression Testing:** Snapshot baseline assertions (toMatchSnapshot()) to prevent breaking changes.
- **5. Performance Testing:** Autocannon 100-virtual user load tests and execution latency SLAs.
- **6. Intelligent Fuzzing:** Exploratory AI boundary fuzzers testing type coercion ("100" vs 100), NaN, and empty bounds.
- **7. Security Testing:** [EXPLOIT PROBE] malicious payloads and [GUARD CHECK] validation assertions.

5. Code Knowledge Graph Schema (Neo4j Graph-RAG)

Nodes & Edge Relationships stored in graph storage:

- **Node Labels:** :File, :Function, :Class, :Endpoint, :Vulnerability, :TestCase
- **Edge Relationships:** (:File)-[:CONTAINS]->(:Function), (:File)-[:IMPORTS]->(:File), (:File)-[:EXPOSES_ROUTE]->(:Endpoint), (:Vulnerability)-[:FOUND_IN_FUNCTION]->(:Function)

6. GitHub Repository & Deployment

- **GitHub Repository:** https://github.com/tharunpraveen/repo_s-t
- **Target Branch:** main
- **Backend API:** <http://localhost:3002> (Express API Server)
- **Frontend UI:** <http://localhost:3003> (Next.js Interactive Dashboard)